

排序算法总结

本讲义总结了常见的六大内部排序算法，包括它们的核心思想、时间复杂度、稳定性与典型代码实现。

插入排序 (Insertion Sort)

算法说明

插入排序通过维护一个“已排序区间”，不断将未排序区间的元素插入到适当位置。逻辑类似于整理扑克牌。

特点：

- **时间复杂度**
 - 最好： $O(n)$
 - 平均： $O(n^2)$
 - 最坏： $O(n^2)$
- **空间复杂度**： $O(1)$
- **稳定性**：稳定
- **适用场景**：数据量小、基本有序

代码实现

```
void InsertRange(int a[], int n)
{
    for (int i = 0; i < n - 1; i++)
    {
        int end = i;
        int temp = a[end + 1];

        while (end >= 0 && a[end] > temp)
        {
            a[end + 1] = a[end];
            end--;
        }

        a[end + 1] = temp;
        arrPrint(a, n);
    }
}
```

希尔排序（Shell Sort）

算法说明

希尔排序基于插入排序，通过“分组+缩小 gap”方式加速排序效率。

- **时间复杂度：** $O(n^{1.3})$ （依赖增量序列）
- **空间复杂度：** $O(1)$
- **稳定性：**不稳定
- **适用场景：**中等规模数据，对稳定性要求不高

代码实现

```
void ShellSort(int a[], int n)
{
    for (int gap = n / 2; gap > 0; gap /= 2)
    {
        for (int i = gap; i < n; i++)
        {
            int temp = a[i];
            int end = i - gap;

            while (end >= 0 && a[end] > temp)
            {
                a[end + gap] = a[end];
                end -= gap;
            }

            a[end + gap] = temp;
        }
    }
}
```

堆排序（Heap Sort）

算法说明

基于 **大顶堆** / **小顶堆** 实现的选择型排序。每次把堆顶（最大值）放到末尾。

- **时间复杂度**： $O(n \log n)$
- **空间复杂度**： $O(1)$
- **稳定性**：不稳定
- **适用场景**：要求 $O(n \log n)$ 且空间有限

代码实现

```
void AdjustDown(int a[], int size, int root)
{
    int parent = root;
    int child = 2 * parent + 1;

    while (child < size)
    {
        if (child + 1 < size && a[child] < a[child + 1])
            child++;

        if (a[child] > a[parent])
        {
            Swap(a + child, a + parent);
            parent = child;
            child = parent * 2 + 1;
        }
        else break;
    }
}

void HeapSort(int a[], int size)
{
    for (int i = (size - 2) / 2; i >= 0; i--)
        AdjustDown(a, size, i);

    arrPrint(a, size);

    int end = size - 1;
    while (end > 0)
    {
        Swap(&a[0], &a[end]);
        AdjustDown(a, end, 0);
        end--;
    }

    arrPrint(a, size);
}
```

选择排序（Selection Sort）

算法说明

选择排序每次从未排序区间选择最小（或最大）元素放到正确位置。

- 时间复杂度： $O(n^2)$
- 空间复杂度： $O(1)$
- 稳定性：不稳定
- 适用场景：小规模数据，简单易实现

代码实现

```
void SelectSort(int a[], int size)
{
    int begin = 0, end = size - 1;

    while (begin < end)
    {
        int mini = begin, maxi = begin;

        for (int i = begin; i <= end; i++)
        {
            if (a[i] < a[mini]) mini = i;
            if (a[i] > a[maxi]) maxi = i;
        }

        Swap(a + begin, a + mini);

        if (begin == maxi)
            maxi = mini;

        Swap(a + end, a + maxi);

        begin++;
        end--;
    }
}
```

快速排序 (Quick Sort)

算法说明

通过选取基准元素，把数组划分为左右两个区间并递归处理。

- **时间复杂度**
 - 最好: $O(n \log n)$
 - 平均: $O(n \log n)$
 - 最坏: $O(n^2)$
- **空间复杂度**: $O(\log n)$ (平均)
- **稳定性**: 不稳定
- **适用场景**: 大规模数据, 高性能需求

代码实现（挖坑法）

```
int PartSort1(int a[], int left, int right)
{
    int begin = left, end = right;
    int pivot = begin;
    int key = a[pivot];

    while (begin < end)
    {
        while (begin < end && a[end] >= key) end--;
        a[pivot] = a[end];
        pivot = end;

        while (begin < end && a[begin] <= key) begin++;
        a[pivot] = a[begin];
        pivot = begin;
    }

    a[pivot] = key;
    return pivot;
}

void QuickSort(int a[], int left, int right)
{
    if (left >= right) return;

    int keyindex = PartSort1(a, left, right);

    QuickSort(a, left, keyindex - 1);
    QuickSort(a, keyindex + 1, right);
}
```

归并排序（Merge Sort）

算法说明

归并排序通过“分治 + 合并”实现稳定、高效的排序。

- 时间复杂度： $O(n \log n)$
- 空间复杂度： $O(n)$
- 稳定性：稳定
- 适用场景：对稳定性要求高、大数据量

代码实现

```
void _MergeSort(int a[], int left, int right, int *temp)
{
    if (left >= right) return;

    int mid = (left + right) >> 1;

    _MergeSort(a, left, mid, temp);
    _MergeSort(a, mid + 1, right, temp);

    int begin1 = left, end1 = mid;
    int begin2 = mid + 1, end2 = right;
    int index = left;

    while (begin1 <= end1 && begin2 <= end2)
    {
        if (a[begin1] <= a[begin2])
            temp[index++] = a[begin1++];
        else
            temp[index++] = a[begin2++];
    }

    while (begin1 <= end1) temp[index++] = a[begin1++];
    while (begin2 <= end2) temp[index++] = a[begin2++];

    for (int i = left; i <= right; i++)
        a[i] = temp[i];
}

void MergeSort(int a[], int n)
{
    int* temp = (int*)malloc(sizeof(int) * n);
    _MergeSort(a, 0, n - 1, temp);
    free(temp);
}
```


排序算法复杂度对比

排序算法	平均时间	最坏时间	最好时间	空间复杂度	稳定性	是否原地
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	✓ 稳定	✓
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	✗ 不稳定	✓
堆排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	✗ 不稳定	✓
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	✗ 不稳定	✓
快速排序	$O(n \log n)$	$O(n^2)$	$O(n \log n)$	$O(\log n)$	✗ 不稳定	✓
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	✓ 稳定	✗